KIIT DEEMED TO BE UNIVERSITY · SCHOOL OF COMPUTER ENGINEERING

AI Shopping Agent

A conversational shopping agent grounded in a live cross-merchant catalogue, designed so that fabricated products are impossible rather than unlikely.

Ridhi Kumari

Roll No. 23052828

B.Tech, Information Technology

KIIT Deemed to be University

THE PROBLEM

Buyers think in intent. Search works in keywords.

— **Translation burden.** The shopper must convert a need into keywords, then filters, then a dozen open tabs.

— **Stale results.** A keyword match can be several hardware generations old, or an accessory rather than the device.

— **Unsupported claims.** “Yes, that works with your Mac” — with nothing behind it.

“something for editing 4K video on a budget”

Conventional search cannot answer this. It has no notion of what the sentence is for.

WHY AN AGENT

Three properties make a single model call insufficient

01

Unknown depth

How many steps are needed is not knowable before the conversation starts.

02

External truth

The answer lives in a catalogue that changes hourly. A recalled price is worse than no price.

03

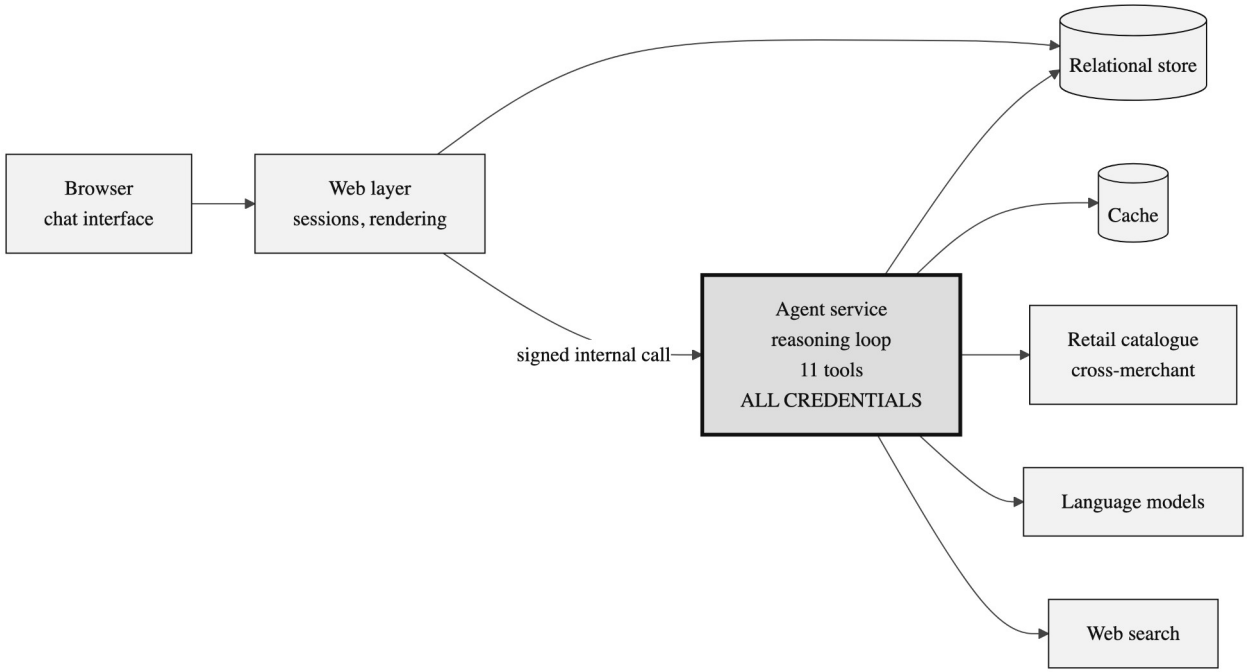
Recoverable failure

Searches come back empty or off-target often. The system must notice and re-query.

So the system plans, calls tools, observes results, corrects itself, and stops on an explicit condition — the standard agent loop.

SYSTEM ARCHITECTURE

Three processes, one trust boundary



The agent service (heavy outline) is the only process holding catalogue, model or search credentials. The web layer proxies everything across an authenticated boundary.

ENGINEERING

What the system is actually built from

Frontend

Next.js 16 App Router

React 19, server components

Tailwind CSS, shadcn/ui, Radix

SWR, next-auth

Agent service

Node.js, strict TypeScript

Vercel AI SDK — streamText

Zod schemas on every tool

Hand-written MCP client

Data

PostgreSQL 16, Drizzle ORM

Redis 7 for feed + evidence

Message parts stored as JSON

SSE response streaming

Verification

Vitest — unit and routes

Playwright — end to end

Scenario harness — behaviour

Static type checking

29 models

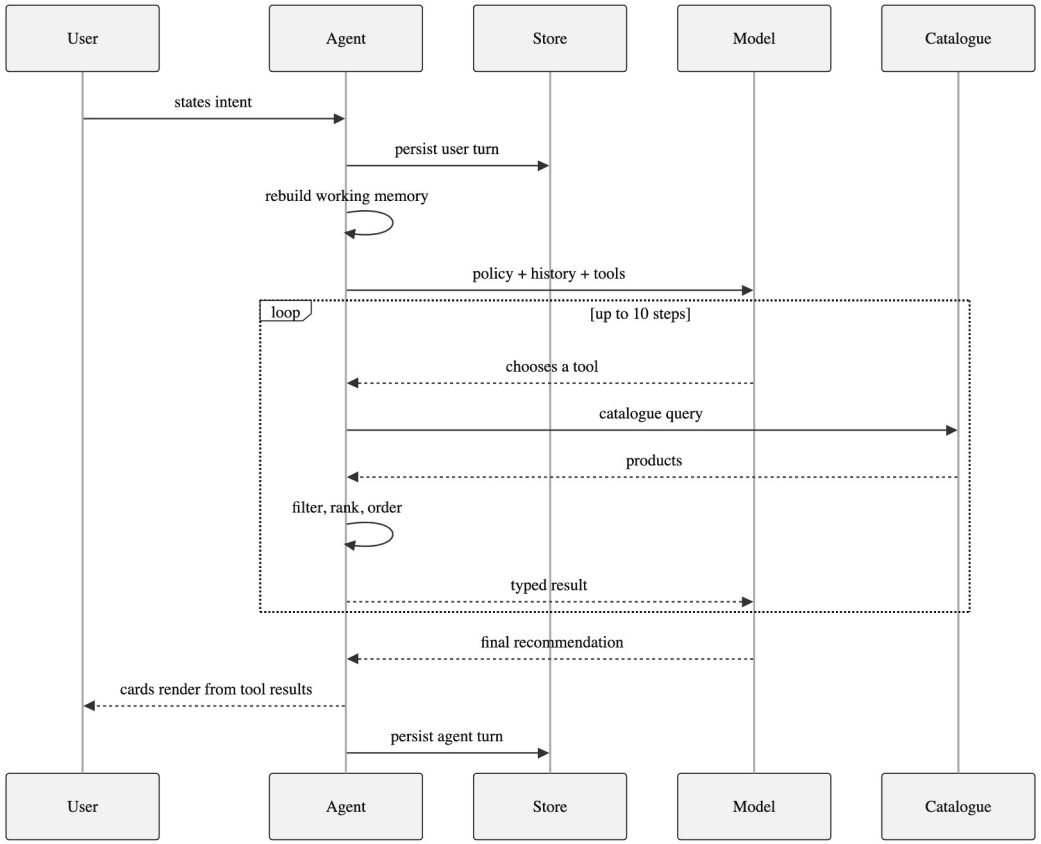
selectable at run time, 22 tool-capable — lets policy failures be separated from model failures

JSON-RPC 2.0

every catalogue call is wrapped in an MCP tools/call envelope with an agent-identity header

ONE TURN, END TO END

Up to ten reasoning steps per message



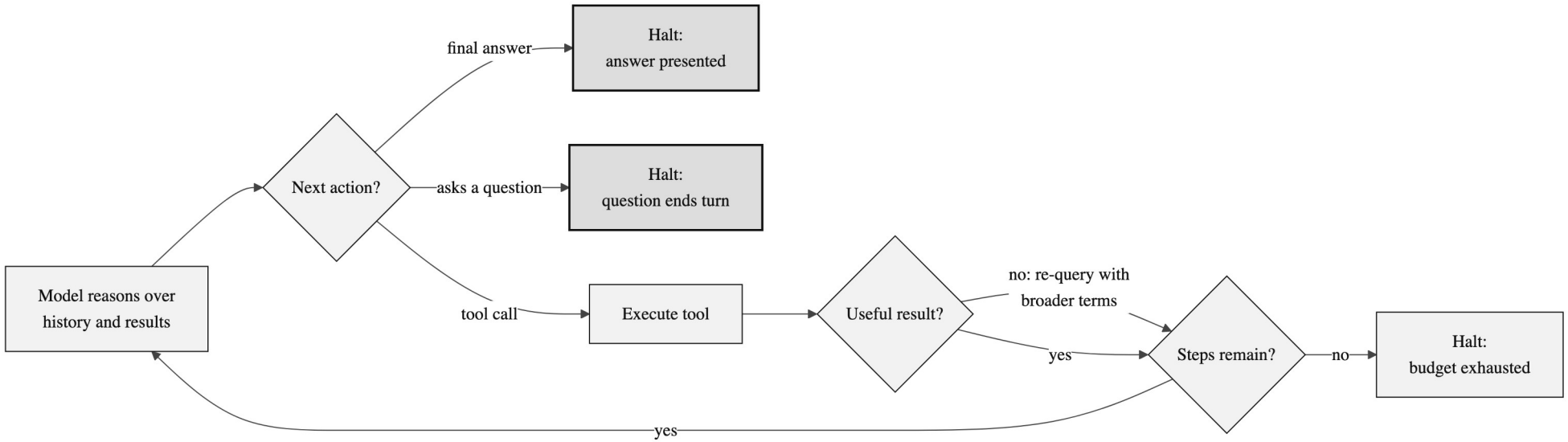
— User message is persisted before the stream opens.

— Working memory is rebuilt from the transcript.

— The boxed region is the agent loop.

— The model sees typed results, never raw network traffic.

Bounded, with two stopping conditions



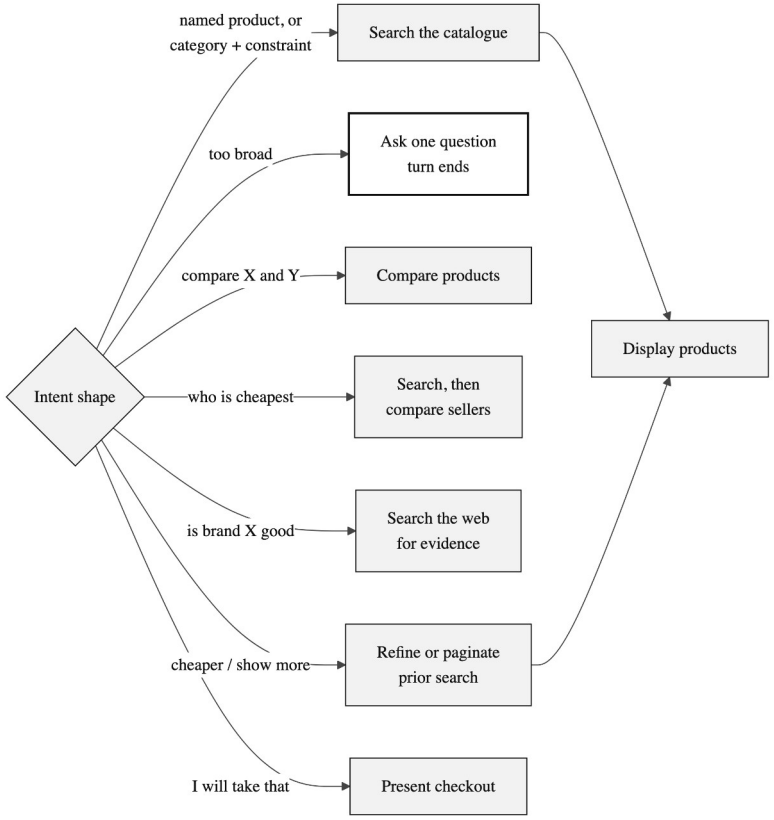
Step budget

Ten steps per turn. Bounds cost; stops a confused model looping on a failing search.

Clarification is terminal

Asking the shopper a question ends the turn — enforced by the runtime, not requested in the prompt.

Eleven typed tools, routed by intent shape



Discovery

Search · Refine · Show more

Presentation

Display products

Analysis

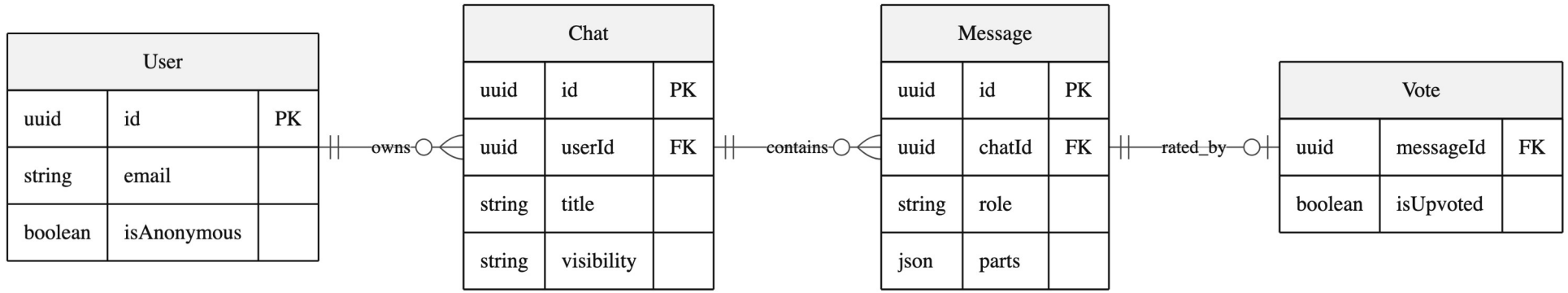
Get product · Compare products · Compare sellers

Interaction

Clarify · Buy · Web search · Web fetch

Search and display are deliberately separate, so the agent can search three times, discard two, and show only the third.

Memory derived from the transcript, not stored beside it



Each message stores an ordered array of parts — text, reasoning, and every tool call with its input and output.

Replayable

Historical chats re-render exactly, because the tool results are still there.

Derived memory

Prior query, filters and seen IDs are rebuilt per request — no separate state to go stale.

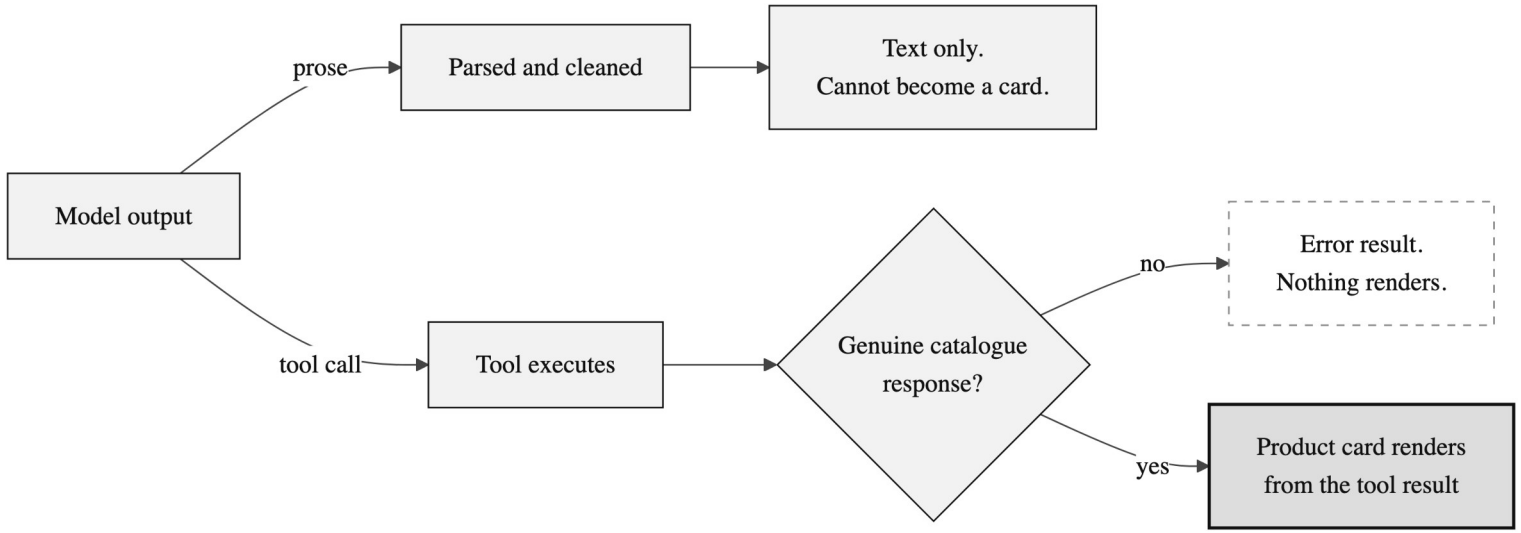
Scoreable

The evaluation harness reads the tool sequence straight out of the transcript.

Caching: feed 1 hour · web evidence 5 minutes · catalogue never — freshness is the point, and the catalogue terms forbid it.

THE CENTRAL IDEA

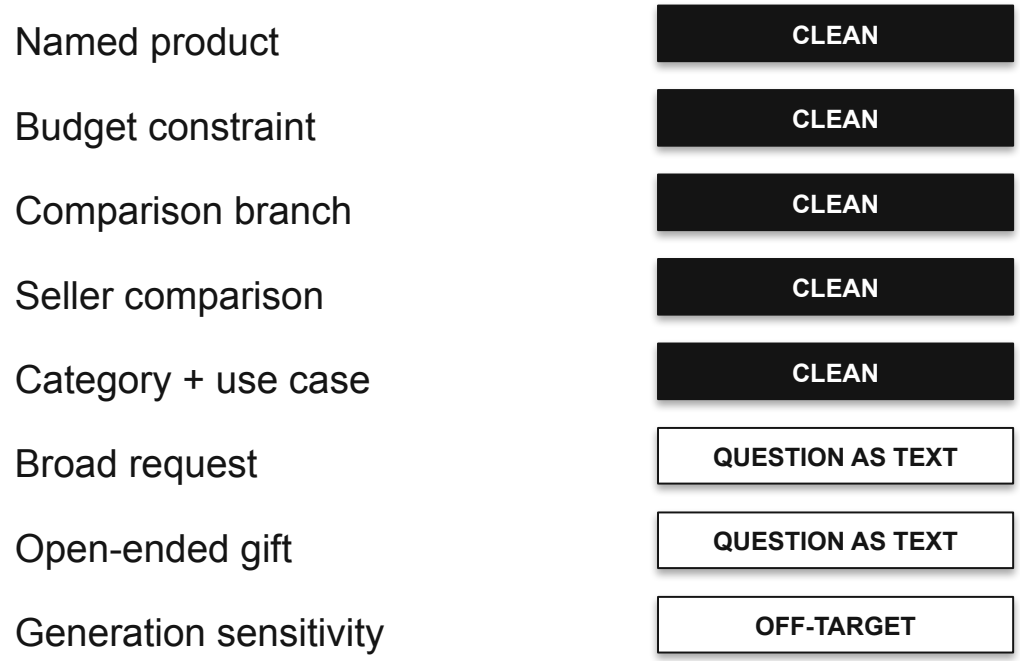
Hallucinated products are impossible, not unlikely



The interface renders product cards only from tool results. The model can ask for a card, but it cannot supply its contents — so inventing a price or a product is not a behaviour it is discouraged from, but one the architecture does not permit.

EVALUATION

Judged on behaviour, not on how the reply reads



5 of 8 scenarios clean

three flagged, each with a diagnosed cause

Two of the three failures produced output that looked entirely correct on screen. Reading the replies would have passed both. Only assertions over the tool sequence caught them.

For agentic systems, behavioural evaluation is the primary instrument, not a supplement.

CONCLUSION

What the project shows, and what comes next

The useful part of an agentic system is not the conversation — it is the constraints around it: the bounded loop, the typed tools, the derived memory, and the grounding contract.

Delivered

— Live cross-merchant catalogue integration

— Eleven typed tools, bounded reasoning loop

— Grounding enforced by the interface contract

— Four-layer verification incl. behavioural harness

Next

— Automatic failover between models

— Constrained output for clarifying questions

— Larger evaluation set covering multi-turn chains

Thank you

Questions welcome

Ridhi Kumari · Roll No. 23052828

School of Computer Engineering · KIIT Deemed to be University